

Assignment - 06

Title : Threaded Binary Tree

Objective : To Study Some advanced data structure
Such as - Threaded Binary Tree.

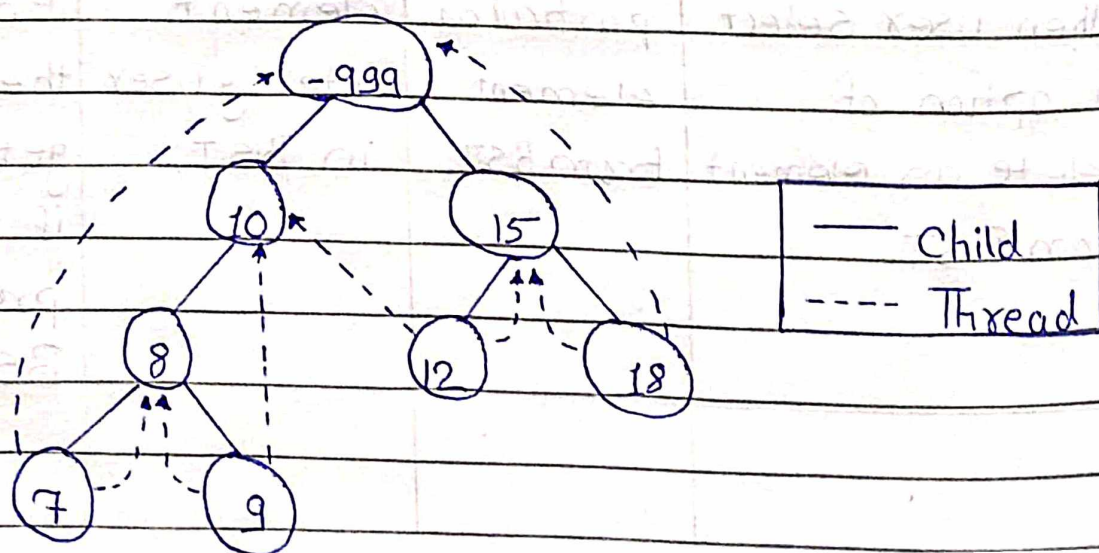
Problem Statement :

Implement In-order Threaded Binary Tree and traverse
it in-Order and preorder.

Outcome :- Solve problem using algorithm design
technique's and data structure.

Theory :

Threaded Binary Tree :- In threaded binary tree the
leaf node's are having the Null value's in the left and
right link fields. It is just a wastage of Memory.
So to avoid the Null value the threads are used.
The threads are nothing but the link's to predecessor
and successor nodes.



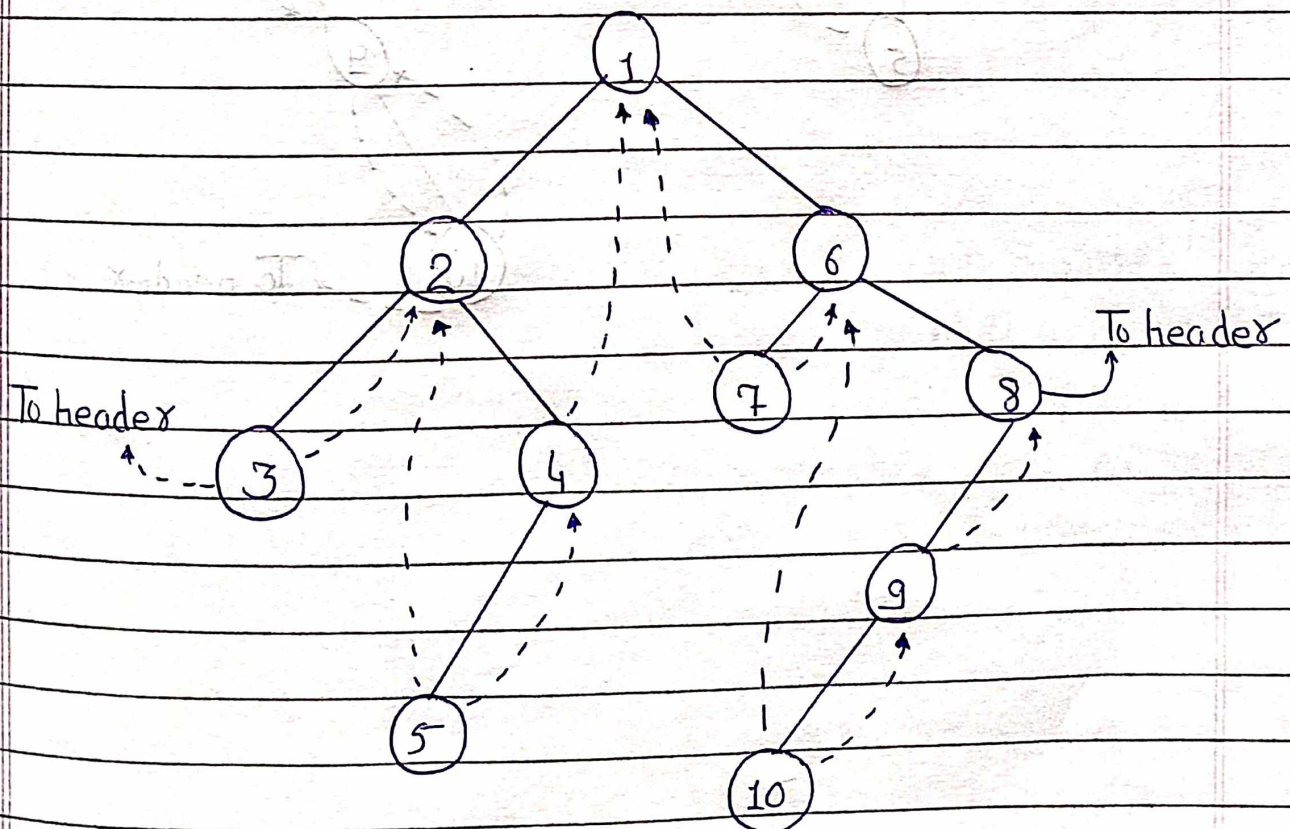
* **Predecessor** :- While displaying the tree, if some particular node occurs previous to some other node than that node is called predecessor of the other node.

* **Successor** :- It is a node which occurs next to some node.

There are three types of threading possible - Inorder, preorder and postorder threading.

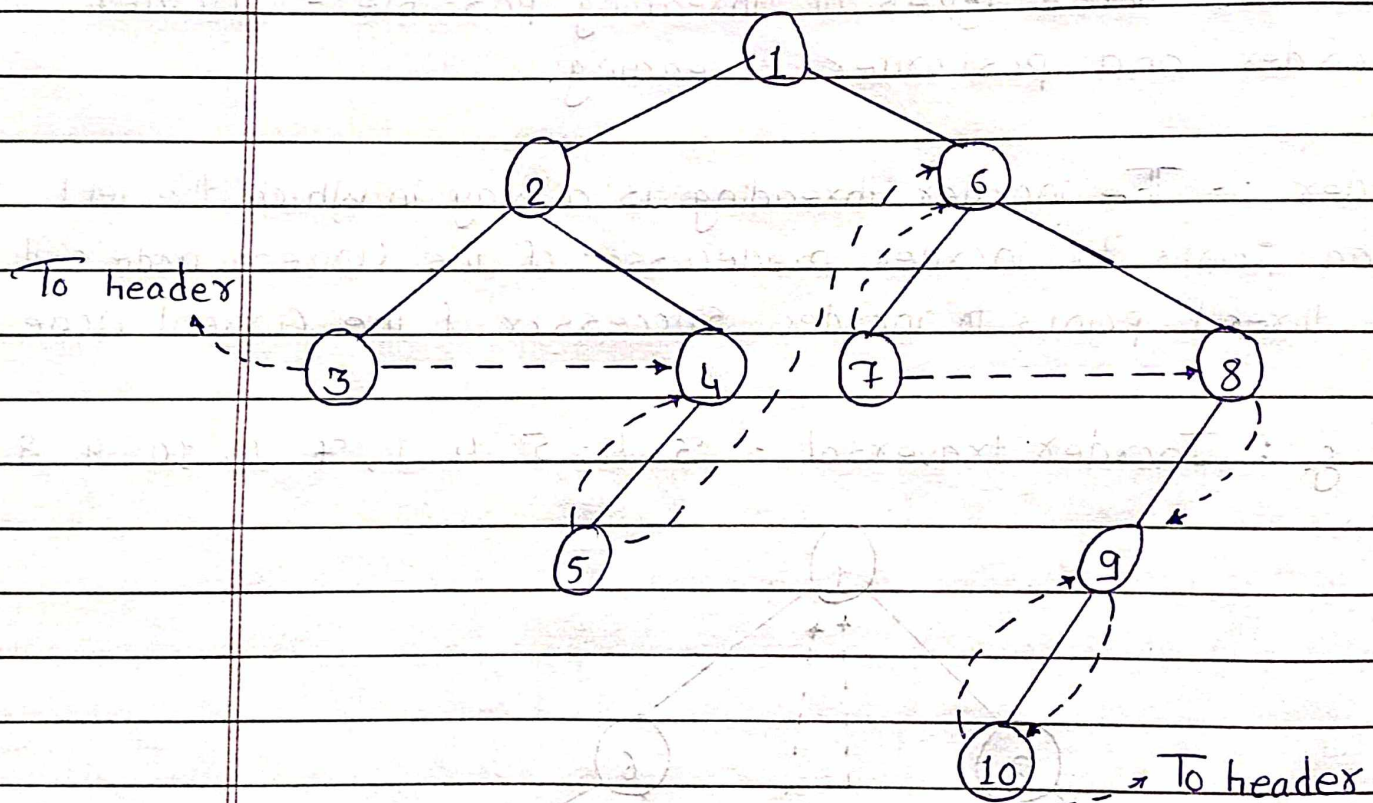
1) **Inorder** :- The inorder threading is a way in which the left thread points to inorder predecessor of the current node and right thread points to inorder successor of the current node.

e.g. : Inorder traversal - 3, 4, 5, 1, 7, 6, 10, 9, 8



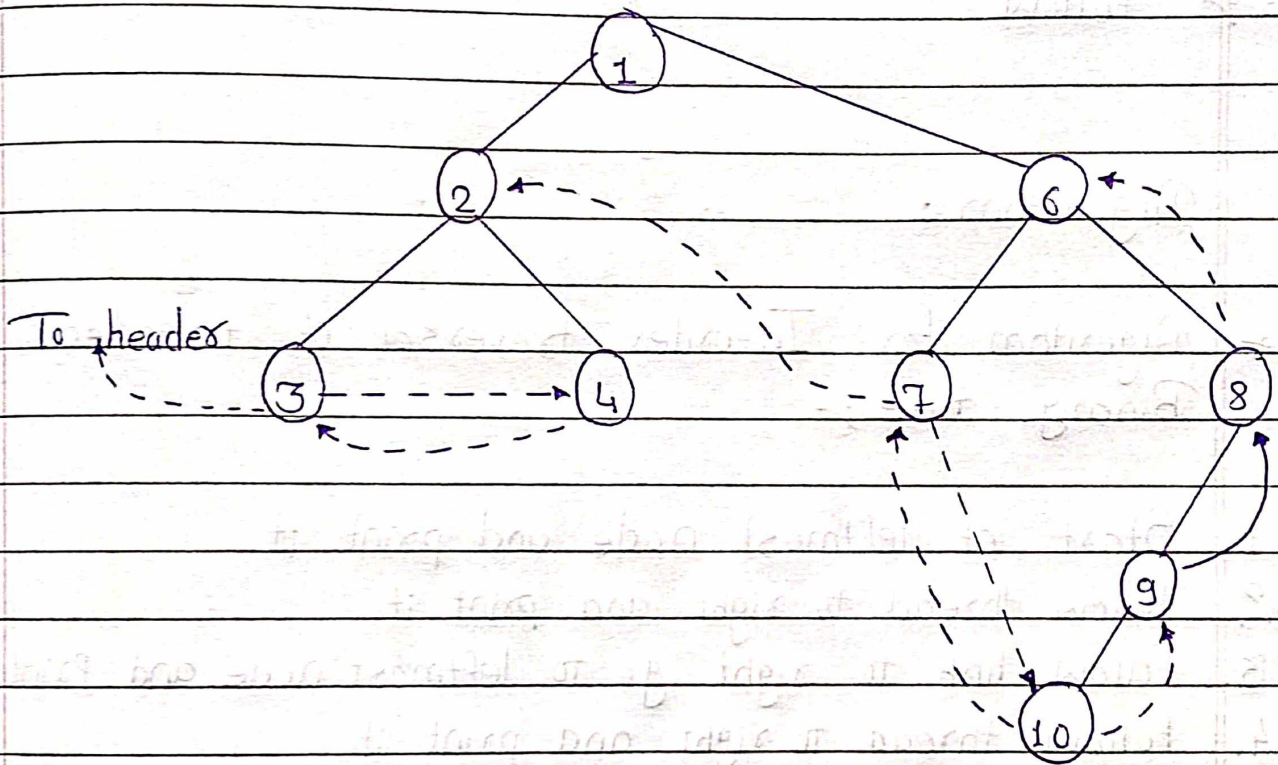
27. preorder :- The preorder threading is a way in which the left thread point to preorder predecessor of the current node and right thread point to preorder successor of it.

e.g : preorder sequence - 1, 2, 3, 4, 5, 6, 7, 8, 9, 10



5). Postorder :- The postorder threading is a way in which the left thread point to postorder predecessor and right points to postorder successor of it.

e.g: Postorder traversal - 3, 4, 2, 7, 10, 9, 8, 6, 1.



Advantage's of threaded Binary tree :-

1. In threaded binary tree there is no Null pointer present, Hence memory wastage in occupying Null link's is avoided.
2. The thread's are pointing to successor and predecessor node. This make's us to obtain predecessor & Successor node of any node quickly.
3. There is no need of Stack while traversing the tree, because using thread links we can reach to previously visited node's.

Algorithm :

* Algorithm for In-order traversal of threaded Binary tree :-

1. Start at leftmost node and print it.
2. Follow thread to right and print it.
3. Follow link to right go to leftmost node and print it.
4. Follow thread to right and print it.
5. (repeat)

* Algorithm for preorder traversal of threaded Binary tree:-

1. Make root node as Current node. until Current node is not Null. print Current node.
2. IF Current node has left child, then make left child as Current node.
3. else if Current node has right child make right child as Current node.
4. else, until right thread exist for Current node
Traverse up the right thread's and update Current.
if Current node = last node STOP.
else make right child of Current node as Current.

Test Case	Test Case Description	Test data	Expected Result	Actual Result	Pass / Fail
1.	Check the Result Where user Select the option of inorder traversal.	Traverse the threaded inorder sequence.	Result Come as inorder traversal of TBT.	TBT get traversal in inorder sequence	pass
2.	check the result When the User select option of pre-order traversal of TBT	Traverse the TBT in Pre-order	preorder traversal of TBT in the pre-order	pre-order traversal of TBT get printed.	pass

Assignment 6

```
#include<iostream>
using namespace std;
struct ITBSTnode
{
    int data;
    int lt,rt;
    ITBSTnode *lc,*rc;
};
class ITBST
{
private:
    ITBSTnode *temp,*newp,*root,*par,*dummy;
public:
    ITBST()
    {
        root=NULL;
    }
    void ITBST_create();
    void ITBST_insert(ITBSTnode *,ITBSTnode *);
    void ITBST_display();
    void inorder_ITBST(ITBSTnode *,ITBSTnode *);
    void preorder_ITBST(ITBSTnode *,ITBSTnode *);

};
void ITBST :: ITBST_create()
{
    newp = new ITBSTnode;
    newp->lc=newp->rc=NULL;
    newp->lt=newp->rt=0;
    cout<<"Enter the value : ";
    cin>>newp->data;

    if(root==NULL)
    {
        root = newp;
        dummy = new ITBSTnode;
        dummy->data = -999;
        dummy->lc = root;
        root->lc = dummy;
        root->rc = dummy;
    }
    else
```

```

{
    ITBST_insert(root,newp);
}

}
void ITBST :: ITBST_insert(ITBSTnode *root, ITBSTnode *newp)
{
    if(newp->data < root->data)
    {
        if(root->lt==0)
        {
            newp->lc=root->lc;
            newp->rc=root;
            root->lc=newp;
            root->lt=1;

        }
        else
        {
            ITBST_insert(root->lc,newp);
        }
    }

    if(newp->data > root->data)
    {
        if(root->rt==0)
        {
            newp->rc=root->rc;
            newp->lc=root;
            root->rt=1;
            root->rc=newp;

        }
        else
        {
            ITBST_insert(root->rc,newp);
        }
    }
}
void ITBST :: inorder_ITBST(ITBSTnode *temp,ITBSTnode *dummy)
{
    while(temp!=dummy)
    {
        while(temp->lt==1)
        {
            temp=temp->lc;

```

```

    }
    cout<<" "<<temp->data;
    while(temp->rt==0)
    {
        temp=temp->rc;
        if(temp==dummy)
            return;
        cout<<" "<<temp->data;
    }
    temp = temp->rc;
}
}

void ITBST :: preorder_ITBST(ITBSTnode *temp,ITBSTnode *dummy)
{
    int flag=0;
    while(temp!=dummy)
    {
        if(flag==0)
        {
            cout<<" "<<temp->data;
        }
        if((temp->lt==1) && (flag==0))
        {
            temp = temp->lc;
        }
        else
        {
            while(1)
            {
                if(temp->rt==1)
                {
                    flag=0;
                    temp=temp->rc;
                    break;
                }
                else
                {
                    if(temp!=dummy)
                    {
                        flag=1;
                        temp=temp->rc;
                        break;
                    }
                }
            }
        }
    }
}

```

```

    }
}
void ITBST :: ITBST_display()
{
    void inorder_ITBST(ITBSTnode *root,ITBSTnode *dummy);
    void preorder_ITBST(ITBSTnode *root,ITBSTnode *dummy);
    ITBST it;

    if(root==NULL)
    {
        cout<<"Inorder threaded Binar search tree has not been created."<<endl;
    }
    else
    {
        cout<<"Inorder Traversal of ITBST - "<<endl;
        it.inorder_ITBST(root,dummy);
        cout<<"\nPreorder Traversal of ITBST - "<<endl;
        it.preorder_ITBST(root,dummy);
    }
}
int main()
{
    int ch;
    ITBST it;
    char ans ='n';

    do
    {
        cout<<"***Inorder Threaded Binary Search Tree***"<<endl;
        cout<<"1.Create ITBST"<<endl;
        cout<<"2.Display ITBST with Inorder and Preorder Traversals"<<endl;
        cin>>ch;

        switch(ch)
        {
            case 1:
                do
                {
                    it.ITBST_create();
                    cout<<"Do you want to enter more elments??(y/n)"<<endl;
                    cin>>ans;
                }
                while(ans=='y');
                break;

```

```
case 2:
it.ITBST_display();
break;

}
cout<<"\nWant to go back to Main Menu??(y/n)"<<endl;
cin>>ans;
}
while(ans=='y');

return 0;
}
```